

SCADA Systems

2026-03-03 – Hans-Petter Halvorsen

1. Introduction

SCADA (Supervisory Control And Data Acquisition) is a type of Industrial Automation and Control System (IACS). Industrial Automation and Control Systems (IACS) are computer systems that control and monitor industrial processes. Industrial Automation and Control Systems, like PLC (Programmable Logic Controller), DCS (Distributed Control System) and SCADA (Supervisory Control And Data Acquisition) share many of the same features. Figure 1 shows the components typically part of a SCADA system.

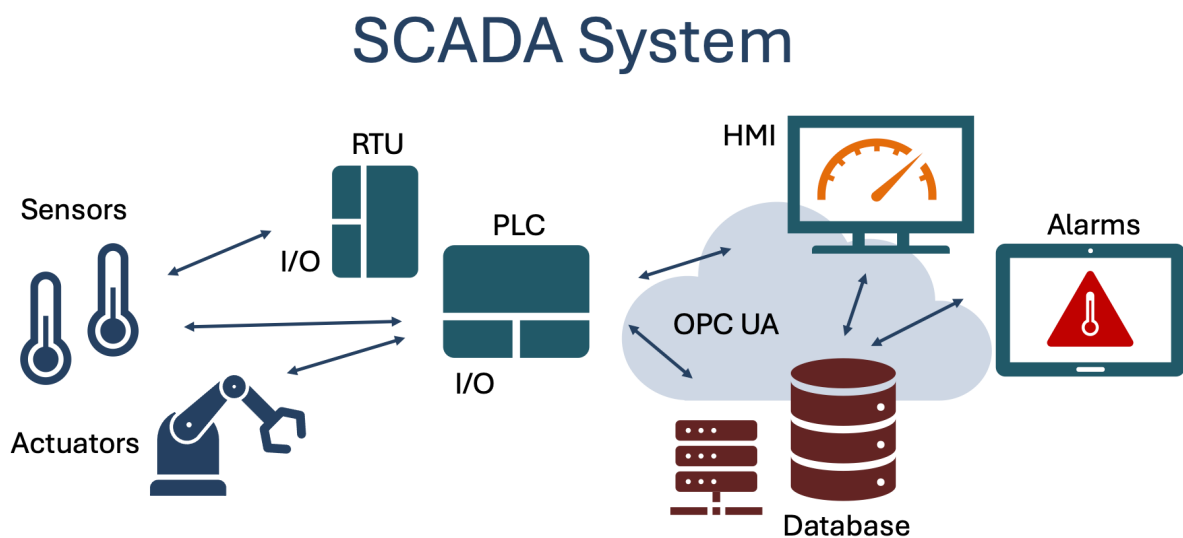


Figure 1: SCADA System Overview

A SCADA system typically contains different modules, such as:

- OPC Server
- A Database that stores all the necessary data
- Control System
- Datalogging System
- Alarm System

These modules are typically implemented as separate applications because they should be able to run on different computers in a network (a so-called distributed system).

2. Background

You work as a system engineer in the R&D department in a system engineering company. **Your Assignment is to develop a next generation SCADA system** in form of a prototype/Proof of Concept (PoC):

- **The system should be module-based and include a Control Module, a Datalogging Module, and an Alarm Module.**

- OPC should typically be used for communication between the different Modules. You can choose between OPC DA and OPC UA.
- Data should be stored in a SQL Server database.
- You need to design a general and flexible database structure that is suitable for the system.
- Creating proper and user-friendly GUI/HMI is an important part of the prototype.
- Note! You can freely choose the programming languages and frameworks to use in the different parts.
- The delivery is a **scientific paper** where you shall give an overview of the entire system made, including the methods used and the results archived.
- The scientific paper shall be published in an international journal in competition with many others, so it is important that you “Add Value” and stand out compared to the others to be selected.
- The scientific papers delivered will be assessed by a committee and only the best contributions will be selected to be published in the international journal.

Figure 2 shows a typical use case scenario for such a SCADA System.

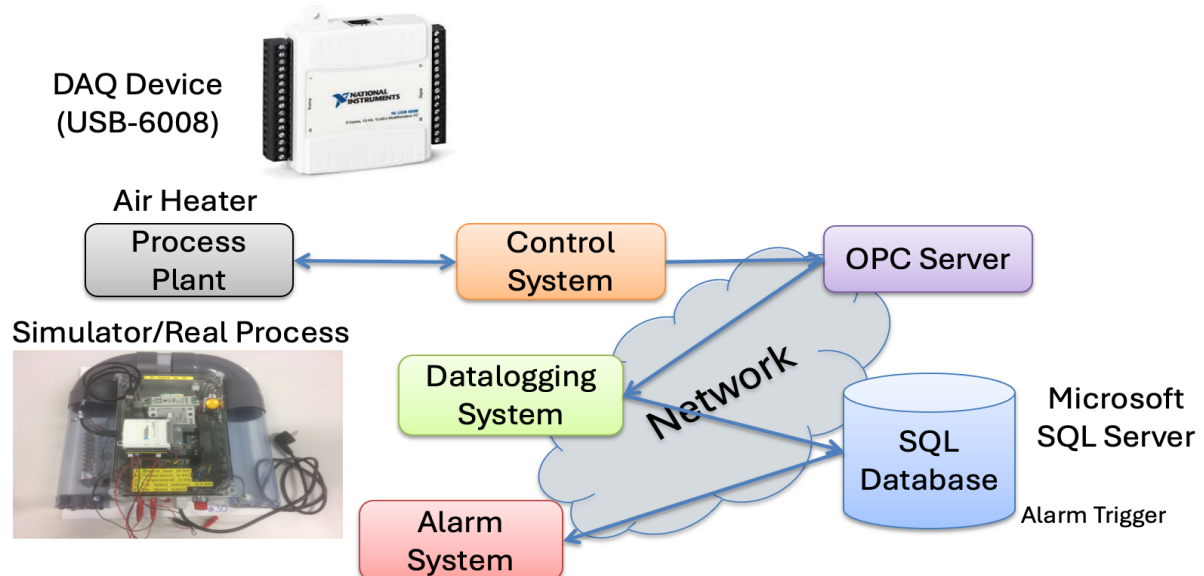


Figure 2: Use case scenario for a typical SCADA System

3. Assignment

Here you see the complete requirements for the SCADA system:

Task 1 – Database

- Design the database used by the SCADA system by creating an ER diagram using a proper ERD tool, like ER Designer.
- The database should store data generated and used by the SCADA system. The database should typically have information about the sensors, the alarms, OPC information, etc. Make sure to create a general and flexible structure.
- Implement the database using SQL Server.

Resources:

- Database Design and Modelling using DB Designer: <https://youtu.be/jcBlzOfIxPs>
- SQL Server and Structured Query Language (SQL): <https://youtu.be/sl6skicZse0>

Task 2 – Control System

- Create a Control System and send data to an OPC Server. Use the **Air Heater** system.
- Start with a model of the system. When the simulations works, use the the real system. **USB-6008** should be used as an interface between the application and the Air Heater.
- You should create and use your own **PI(D)** controller and **lowpass filter** from scratch. Make sure to tune and test your PI(D) controller properly.
- Write data to an OPC Server (you can choose between OPC DA or OPC UA).
- The Control System is typically implemented as a Windows Desktop Application, you can e.g., use LabVIEW or Visual Studio/C#.

Resources:

- **Discrete Control Systems in LabVIEW:** <https://youtu.be/rqKdInEqXdo>
- **Simulation and Control with C# and WinForms:** <https://youtu.be/l6Mq79Dai7M>
- DAQmx in LabVIEW: <https://youtu.be/zk-Z59Gn9aw>
- Visual Studio/C# and DAQ: <https://youtu.be/2s7jHUoXKFY>
- Plotting Data in Windows Forms using ScottPlot: https://youtu.be/HVH_vNAn8hs

Task 3 – Datalogging System

- Create a Datalogging System that reads data from the OPC Server and log the data to an SQL Server database.
- The Datalogging System is typically a Windows Desktop Application, you can e.g., use LabVIEW or Visual Studio/C#.
- You may want to create and use one or more Stored Procedures that's saves the necessary data to the database.

Resources:

- OPC UA in LabVIEW: <https://youtu.be/AoneC2cSzto>
- OPC UA in Visual Studio/C#: <https://youtu.be/KCW23eq4auw>
- Database Communication in LabVIEW using LabVIEW SQL Toolkit: <https://youtu.be/OMFWWXxde-Y>

Task 4 – Alarm System

- Create an Alarm Generation and Monitoring System. The Alarm System can either be a Windows Desktop Application (created with, e.g., LabVIEW or Visual Studio/C#) or a Web Application (using, e.g., ASP.NET Core).
- You may want to create and use a database Trigger for generating and storing alarms into the Database.
- It should be possible to see generated Alarms and Acknowledge Alarms.

Resources:

- SQL Server with C# Windows Forms App: <https://youtu.be/rXugzELsQl0>
- Plotting Data in Windows Forms using ScottPlot: https://youtu.be/HVH_vNAn8hs
- ASP.NET Core - Get Data from Database: <https://youtu.be/pChkbx9BFVA>
- ASP.NET Core – Charts: <https://youtu.be/mksUls9fx-Q>

Task 5 – Cyber Security

- Deal with and give an overview of relevant Cyber Security issues within your system.

Final System

The different subsystems should typically be implemented as separate applications because they should be able to run on different computers in a network (distributed). You should try to install some of the applications made on different computers in a network and see if communication between them works.

Integration with other Systems? Feel free to integrate your solution with existing industrial software like OPC software, PLC software or SCADA software, e.g., Ignition. Ignition is an example of industrial SCADA software from “Inductive Automation”. You can download an unlimited trial version or the Ignition Maker Edition.

Note! The listed modules and requirements are just a starting point. Feel free to adjust and make improvements. The tasks described in the assignment are somewhat loosely defined and more like guidelines, so feel free to interpret the tasks in your own way with a personalized touch.

Feel free to explore, add value, creativity and think outside the box when it comes to working with the system to be implemented.

Make sure to make user-friendly and intuitive applications with good user interfaces and high code quality. Make sure to include relevant information like units and a proper number of decimals whenever you deal with numbers, etc.

Think about the assignment as a small real-life industrial project, and not a set of tasks or exercises. What does the company that hire you expect from you when you deliver this project? What kind of quality is expected? Try to see your work in a larger context than just an assignment or a set of exercises. Try to see the big picture. The tasks within the assignment are just small building blocks that should end up with a fully working system.

It is the final fully working system that shall be documented, not all tiny details or things that you have done along the way.